



CORE COMPUTER SCIENCE

Operating Systems

Placement Preparation Notes

Created by Pranav Gawai

Operating Systems Roadmap

PLACEMENT PREPARATION GUIDE

How to use this Handbook:

Operating system concepts are heavily tested using comparative questions (Mutex vs Semaphore, Paging vs Segmentation). Focus on how the kernel coordinates system calls, context switching, RAM paging boundaries, and deadlock prevention states.

Three-Phase Learning Plan

PHASE 1

Scheduling & Memory

Master CPU scheduling algorithms, processes vs threads, paging, segmentation, and virtual swapping. (Topics 1 – 8)

PHASE 2

Synchronization & Deadlock

Deep-dive into mutexes, semaphores, classic dining philosophers solutions, and Banker's deadlock safety checks. (Topics 9 – 14)

PHASE 3

Architecture & Storage

Study inodes, disk sweeps, system call interrupts, IPC, privileged CPU modes, and dynamic linker stubs. (Topics 15 – 25)

Target Completion: 2 Hours Core Study & 30 Mins Self-Recall

Created by **Pranav Gawai**
grindos.pranavx.in

Table of Contents

Operating Systems · Placement Preparation Handbook

PROCESS MANAGEMENT

01	Process vs Thread	p.04
02	CPU Scheduling Algorithms	p.05
03	PCB & Context Switching	p.06
04	Multi-Level Queue & Feedback Queue	p.07

MEMORY MANAGEMENT

05	Paging & Segmentation	p.08
06	Virtual Memory & Demand Paging	p.09
07	Page Replacement Algorithms	p.10
08	Page Fault & Thrashing	p.11
20	Fragmentation (Internal vs External)	p.23
21	TLB (Translation Lookaside Buffer)	p.24

PROCESS SYNCHRONIZATION

09	Critical Section Problem	p.12
10	Mutex vs Semaphore vs Monitor	p.13
11	Classic Synchronization Problems	p.14

DEADLOCKS

12	Deadlock: Four Conditions	p.15
13	Banker's Algorithm	p.16
14	Deadlock Detection & Recovery	p.17

FILE SYSTEMS & STORAGE

15	File Systems & Inodes	p.18
16	Disk Scheduling Algorithms	p.19

OPERATING SYSTEM ARCHITECTURE

17	Monolithic vs Microkernel	p.20
18	System Calls (fork, exec)	p.21
19	Inter-Process Communication	p.22

OPERATING SYSTEM SECURITY

22	User Mode vs Kernel Mode	p.25
----	--------------------------	------

OPERATING SYSTEM EXECUTION

23	Dynamic vs Static Linking	p.26
----	---------------------------	------

OPERATING SYSTEM OPERATIONS

24	Spooling vs Buffering	p.27
----	-----------------------	------

OPERATING SYSTEM ARCHITECTURES

25	RTOS vs GPOS	p.28
----	--------------	------

PROCESS MANAGEMENT

Yield: ★★★★★

01 – Process vs Thread

Understanding active execution units and memory allocation contexts.

CORE DIFFERENCES

Process: An executing program instance. Has its own private address space (code, data, heap, file descriptors). Heavyweight, slow creation.

Thread: A subset of a process (lightweight process). Shares code, data, and OS resources with sibling threads, but has its own Stack and Registers.

Process Memory Boundary

Shared: Code | Data | Heap | Files

Thread 1
Stack & RegsThread 2
Stack & Regs

WHERE USED IN INDUSTRY

Web servers like Nginx use a multi-process architecture for isolation, while database engines like MySQL use a multi-threaded approach to share memory buffers efficiently.

INTERVIEW DEPTH

- L1:** Process vs Thread isolation models.
- L2:** Memory differences (private stack vs shared heap).
- L3:** Process/Thread Control Block kernel allocations.
- L4:** TLB preservation differences during thread switches.
- L5:** User-space green threads vs kernel-level scheduling mapping.

CORE INTERVIEW QUESTION

"Why is context switching between threads faster than between processes?"

KEYWORDS TO DROP

VIRTUAL MEMORY SPACE

TLB CACHE FLUSH

SHARED ADDRESS SPACE

CACHE HOTNESS

MODEL ANSWER

"Process context switching requires flushing the CPU TLB cache and loading a completely new page table directory. Thread switching preserves the virtual memory layout, meaning the TLB cache does not need to be flushed and cache lines stay warm, reducing overhead."

DEEP DIVE FOLLOW-UP

→ **"What is a zombie process?"**

A process that has finished execution but still has an entry in the process table so its parent can read its exit status code.

INTERVIEW DEPTH PROGRESSION

Level 1 (Basic): Contrast process isolation with thread resource sharing.

Level 2 (PDU): Detail thread stacks vs shared process heaps.

Level 3 (Switching): Explain why thread switching avoids TLB flushes.

Level 4 (Design): Explain thread pool architecture and worker queue limits.

COMMON MISTAKE

Saying threads don't share memory. They share heap but have private stacks.

TOP FOLLOW-UPS

- What is a thread pool?
- Differentiate user-level vs kernel-level threads.

INTERVIEWER TRAP

Don't say threads share stack space! Threads share heap and code, but each thread **MUST** have its own independent stack to track function call execution.

MEMORY TRICK

Process = Independent houses (isolated yard). Thread = Rooms inside the same house (sharing kitchen, private bedrooms).

PROCESS MANAGEMENT

Yield: ★★★★★

02 – CPU Scheduling Algorithms

Comparing FCFS, SJF, SRTF, and Round Robin scheduling paradigms.

ALGORITHM METRICS

Used to decide which ready process gets CPU time next. Crucial metrics are **Turnaround Time** (completion – arrival) and **Waiting Time** (turnaround – burst).

Algo	Type	Metric	Downside
FCFS	Non-Prem	Arrival	Convoy Effect
SJF	Non-Prem	Burst Time	Starvation of Long
SRTF	Preemptive	Rem Time	High Overhead
RR	Preemptive	Time Quantum	Quantum Sensitive

WHERE USED IN INDUSTRY

Linux's Completely Fair Scheduler (CFS) uses red-black trees to schedule process execution, balancing interactive desktop threads and background database workers.

INTERVIEW DEPTH

- L1: Preemptive vs non-preemptive scheduling algorithms.
- L2: Starvation prevention using aging schemes.
- L3: Multi-level feedback queue parameter calibration.
- L4: Linux CFS red-black tree scheduler mechanics.
- L5: Affinity scheduling on multi-core symmetric multiprocessing.

CORE INTERVIEW QUESTION

"What is the 'Convoy Effect' in FCFS scheduling, and how can it be resolved?"

KEYWORDS TO DROP

FCFS NON-PREEMPTIVE

SHORT VS LONG JOB

AVERAGE WAITING TIME

PREEMPTIVE SRTF

MODEL ANSWER

"The Convoy Effect occurs in FCFS when one CPU-bound process with a huge burst time blocks many short I/O-bound processes, increasing the average waiting time. It is resolved by using preemptive algorithms like Round Robin or Shortest Remaining Time First (SRTF)."

DEEP DIVE FOLLOW-UP

→ "What determines the optimal time quantum in Round Robin?"

If too large, RR degrades to FCFS. If too small, context switching overhead dominates CPU cycles. The sweet spot is context switch overhead < 10% of quantum.

REAL INTERVIEW SCENARIO

Interviewer: "Why is preemptive scheduling used in interactive systems?"

Candidate: "To guarantee responsiveness. If a long process runs non-preemptively, user inputs will lag. Preemption enforces time-slicing (e.g., Round Robin), ensuring CPU access for GUI tasks."

COMMON MISTAKE

Confusing job scheduler with CPU scheduler. CPU scheduler picks from ready queue.

TOP FOLLOW-UPS

- What is convoy effect in FCFS?
- How is time quantum chosen in Round Robin?

INTERVIEWER TRAP

Don't confuse Turnaround Time (TAT) with Waiting Time (WT). TAT is total lifecycle length; WT is the time process spent waiting in ready queue.

MEMORY TRICK

FCFS = Traditional bank teller queue. Round Robin = Bouncing a mic between karaoke singers for 10 seconds each.

PROCESS MANAGEMENT

Yield: ★★★★★☆

03 – PCB & Context Switching

Storing state metadata and transitioning execution across processes.

PCB DATA STRUCTURE

The Process Control Block (PCB) is an OS kernel data structure containing all metadata about a process: PID, state, Program Counter (PC), CPU registers, memory limits, open file descriptors.

PCB Structure Block:

- Process ID (PID)
- Process State (New, Ready, Running...)
- PC Pointer & Registers
- CPU Scheduling Info
- Memory Management Map

WHERE USED IN INDUSTRY

Hypervisors (like KVM or VMware ESXi) swap virtual CPU registers in physical CPU cores during guest VM context switches, adding virtualization overhead.

INTERVIEW DEPTH

- L1: PCB structure: PID, Program Counter, registers, files.
- L2: Context switch register save/restore lifecycle.
- L3: TLB invalidation and CPU cache invalidation costs.
- L4: Hardware-level CPU register window swapping.
- L5: Interrupt service routine execution context switching.

CORE INTERVIEW QUESTION

"Explain the exact lifecycle steps of a CPU during a context switch from Process A to Process B."

KEYWORDS TO DROP

SAVE REGISTER STATE

PROGRAM COUNTER PTR

PCB LOAD / STORE

INTERRUPT HANDLER

MODEL ANSWER

"During a context switch: 1. CPU triggers an interrupt (timer or system call). 2. OS saves the current registers, stack pointer, and Program Counter (PC) into Process A's PCB. 3. OS updates Process A's state. 4. OS schedules Process B, loads its saved registers and PC from Process B's PCB, and updates execution."

DEEP DIVE FOLLOW-UP

→ "Is context switching a pure overhead?"

Yes. During context switching, the CPU performs no productive user-space computation; it is purely executing kernel setup code.

CPU CONTEXT DEPTH

Level 1 (State): Explain that PCB stores process ID, program counter, and state.

Level 2 (Mechanism): Describe CPU register backup to PCB during interrupts.

Level 3 (Overhead): Explain why TLB miss penalty occurs post context switch.

Level 4 (Hardware): Discuss hardware-assisted context switching with register windows.

COMMON MISTAKE

Thinking context switching has zero cost. It consumes CPU cycles and flushes cache.

TOP FOLLOW-UPS

- What is the ready queue storage structure?
- What triggers a context switch?

INTERVIEWER TRAP

Don't state that context switching occurs inside user space. It is strictly a privileged Kernel-mode execution.

MEMORY TRICK

PCB is the save state slot of a video game emulator. Context switch is saving Game A and loading Game B.

PROCESS MANAGEMENT

Yield: ★★★★★

04 – Multi-Level Queue & Feedback Queue

Managing complex workloads by separating processes by characteristics.

QUEUE PARTITIONING

Multi-Level Queue (MLQ): Partition ready queue into separate queues (interactive vs batch). Processes belong permanently to a queue.

Feedback Queue (MLFQ): Allows processes to move between queues based on execution history (prevents starvation, optimizes interactive response).

```
[High Priority (Interactive)] Quantum=4ms
  ↓ processes exceeding quantum demoted
[Medium Priority (Interactive)] Quantum=8ms
  ↓ demoted
[Low Priority (Batch)] FCFS
```



WHERE USED IN INDUSTRY

Real-time operating systems (RTOS) in automotive engine control units (ECUs) segregate critical safety-critical tasks from telemetry tasks in prioritized queues.



INTERVIEW DEPTH

- L1: Multi-Level Queue partition boundaries.
- L2: Priority queues scheduling configurations.
- L3: Feedback Queue aging dynamically migrating processes.
- L4: Fixed priority preemption with time-sliced background queues.
- L5: Solving priority inversion using priority inheritance protocol.

CORE INTERVIEW QUESTION

"How does a Multi-Level Feedback Queue (MLFQ) dynamically prioritize interactive tasks over CPU-bound tasks?"

QUEUE DEMOTION

I/O BURST PRIORITY

AGING MECHANISM

CPU HOG THROTTLING

MODEL ANSWER

"MLFQ starts all new processes at the highest priority queue with a short time quantum. If a process yields before its quantum (interactive I/O), it stays high. If it hogs the CPU for the entire quantum, it gets demoted to a lower priority queue with a longer quantum, keeping interactive response times low."

DEEP DIVE FOLLOW-UP

→ **"What is 'Aging' in scheduling?"**

A safety valve mechanism where processes sitting in low-priority queues for a long time are promoted to higher queues to prevent CPU starvation.

REAL INTERVIEW SCENARIO

Interviewer: "What is the key difference between MLQ and MLFQ?"

Candidate: "MLQ has static queues where processes stay in their assigned queues. MLFQ allows processes to move dynamically between queues based on CPU burst history, solving starvation via aging."



COMMON MISTAKE

Saying processes can move between MLQ queues. That requires Multilevel Feedback Queue (MLFQ).



TOP FOLLOW-UPS

- What is the aging mechanism?
- How does feedback queue scheduling work?



INTERVIEWER TRAP

Don't confuse MLQ with MLFQ. In MLQ, a process is locked to a single queue (e.g. system tasks queue). Only MLFQ supports queue migrations.



MEMORY TRICK

MLFQ is like a corporate trial period: start at top; do good work, stay; take too long, demoted.

MEMORY MANAGEMENT

Yield: ★★★★★

05 – Paging & Segmentation

Analyzing non-contiguous virtual memory mapping paradigms.

LOGICAL SPACE LAYOUTS

Paging: Divides physical memory into fixed-sized blocks (frames) and virtual memory into pages. Eliminates external fragmentation. Causes internal fragmentation.

Segmentation: Divides memory into logical blocks of variable size (main program, stack, library functions). Causes external fragmentation.

Metric	Paging	Segmentation
Block Size	Fixed Size	Variable Size
Mapping	Page Table	Segment Table
Fragmentation	Internal	External
User Visibility	Transparent	Visible (logical blocks)

WHERE USED IN INDUSTRY

High-performance search engines (like Elasticsearch) configure JVM heap allocations using custom memory mapping (mmap) flags to avoid fragmentation latency.

INTERVIEW DEPTH

- L1: Contiguous memory allocation schemes (First/Best/Worst Fit).
- L2: Internal vs External fragmentation causes.
- L3: Dynamic memory compaction and relocation overhead.
- L4: Linux slab/buddy allocator page distribution.
- L5: Process address space virtual layout sections.

CORE INTERVIEW QUESTION

"Why does Paging completely eliminate External Fragmentation but introduce Internal Fragmentation?"

KEYWORDS TO DROP

FIXED PAGE SIZE

LAST PAGE REMAINDER

NON-CONTIGUOUS FRAMES

INTERNAL SLACK SPACE

MODEL ANSWER

"Paging splits memory into fixed frames. Any page can map to any frame, so no small unallocated holes (external fragmentation) are created. However, if a process requests 5KB and page size is 4KB, it gets 2 pages (8KB). The remaining 3KB of the second page is wasted, creating internal fragmentation."

DEEP DIVE FOLLOW-UP

→ "What is a page table, and where does it live?"

A table mapping page numbers to frame numbers. It lives in main memory (RAM) and its physical address is pointed to by a register (PTBR).

ALLOCATION STRATEGY TRADE-OFFS

First Fit: Fastest search; allocates the first block that fits. Leaves fragments at the beginning.

Best Fit: Searches entire list to find smallest block that fits. Minimizes leftover size, but creates tiny useless fragments.

Worst Fit: Allocates largest available block. Leaves largest leftover block, which is reusable.

COMMON MISTAKE

Thinking Best Fit is always best. It leaves useless tiny fragments.

TOP FOLLOW-UPS

- What is internal fragmentation?
- What is dynamic storage allocation?

INTERVIEWER TRAP

Don't say paging is user-visible. Paging is entirely managed by the operating system kernel and Memory Management Unit (MMU) hardware.

MEMORY TRICK

Paging = Slicing a bread loaf into identical slices. Segmentation = Separating a meal into main, salad, dessert.

MEMORY MANAGEMENT

Yield: ★★★★★

06 – Virtual Memory & Demand Paging

Running processes larger than physical RAM via swap mapping.

ADDRESS ABSTRACTION

Virtual memory maps a process's virtual addresses to physical RAM, abstracting disk storage as a virtual expansion of physical RAM.

Demand Paging: Lazy swapper paradigm. A page is loaded into main memory only when a process references it.

```
[Virtual Address Space]
↓ Page Table Lookup (Valid Bit = 0)
[PAGE FAULT INTERRUPT]

↓ OS retrieves page from Swap Space on Disk
[Frame Loaded into RAM] (Valid Bit → 1)
```

WHERE USED IN INDUSTRY

Cloud hypervisors overcommit RAM by mapping guest physical memory pages to swap space on NVMe drives, optimizing server hardware utilization.

INTERVIEW DEPTH

- L1: Virtual-to-physical address space mapping.
- L2: Paging vs Segmentation structural boundaries.
- L3: Page table directories structure (Multi-level page tables).
- L4: Virtual page eviction flags (dirty, reference, present).
- L5: Memory overcommit configurations and OOM killer rules.

CORE INTERVIEW QUESTION

"Step-by-step, describe what happens inside the OS and CPU when a Page Fault occurs."

KEYWORDS TO DROP

VALID/INVALID BIT

PAGE FAULT EXCEPTION

SWAP SPACE LOOKUP

I/O DISK BLOCK

MODEL ANSWER

"When a process requests an address, the MMU checks the page table. If the valid bit is 0, the MMU traps to the OS (Page Fault). The OS: 1. Pauses the process. 2. Locates the missing page on the swap disk. 3. Finds a free physical frame. 4. Reads the disk block into the frame. 5. Updates the Page Table bit to 1. 6. Restarts the instruction."

DEEP DIVE FOLLOW-UP

→ "What is a dirty bit?"

A bit in the page table indicating if the page was modified. If not modified (dirty bit = 0), the OS can discard it during swap out without writing back to disk.

MEMORY DEPTH PROGRESSION

Level 1 (Concept): Virtual Memory provides contiguous private address space.

Level 2 (Paging): Maps virtual pages to non-contiguous physical frames via Page Tables.

Level 3 (MMU): Hardware MMU translates addresses using TLB cache queries.

Level 4 (Multi-Level): Hierarchical page tables save RAM but increase access RTTs.

COMMON MISTAKE

Confusing internal and external fragmentation. Paging causes tiny internal fragmentation; segmentation causes external.

INTERVIEWER TRAP

Don't think virtual memory makes processes run faster. It increases overhead. It simply allows running larger processes than physical memory size allows.

TOP FOLLOW-UPS

- What is copy-on-write (COW)?
- How large is a standard page table?

MEMORY TRICK

Virtual memory is like display books in a bookstore window: you can view the cover, but the salesman brings the book from the back only when asked.

MEMORY MANAGEMENT

Yield: ★★★★★

07 – Page Replacement Algorithms

Deciding which frames to evict from physical memory.

EJECTION PARADIGMS

FIFO: First In First Out. Simplest, suffers from Belady's Anomaly.

LRU (Least Recently Used): Replaces the page unused for the longest period. High tracking overhead.

Optimal (MIN): Replaces page that won't be used for the longest future duration. Impossible to implement (requires future knowledge).

Belady's Anomaly (FIFO)

Increasing physical frames actually causes MORE page faults for specific reference strings (e.g. 1,2,3,4,1,2,5,1,2,3,4,5).

WHERE USED IN INDUSTRY

Content Delivery Networks (CDNs) like Cloudflare use Least Recently Used (LRU) variants to cache assets, evicting cold assets from edge node RAM.

INTERVIEW DEPTH

- L1: Page replacement algorithms: FIFO, LRU, Optimal.
- L2: Belady's Anomaly triggering on FIFO replacement.
- L3: LRU approximation using reference bits (Clock algorithm).
- L4: Page buffering and dirty page flushing strategies.
- L5: Working set model and page reference string analysis.

CORE INTERVIEW QUESTION

"What is Belady's Anomaly? Name the page replacement algorithms that never suffer from it."

KEYWORDS TO DROP

FIFO QUEUE

FRAME INCREASE FAULTS

STACK ALGORITHMS

LRU REFERENCE MAP

MODEL ANSWER

"Belady's Anomaly is a counter-intuitive phenomenon where increasing the number of physical memory frames causes an increase in page faults under FIFO. 'Stack' algorithms, such as LRU and Optimal, can never suffer from Belady's anomaly, as their frame allocation subset holds inclusion properties."

DEEP DIVE FOLLOW-UP

→ "How is LRU implemented in real systems?"

Due to the high overhead of tracking memory accesses, OS kernels use approximations like the **Second Chance (Clock)** algorithm using reference bits.

REAL INTERVIEW SCENARIO

Interviewer: "What is Belady's Anomaly and when does it occur?"

Candidate: "Belady's Anomaly is when a page replacement algorithm causes more page faults as physical memory frames increase. It occurs in FIFO page replacement, but never in stack-based algorithms like LRU."

COMMON MISTAKE

Stating LRU is mathematically optimal. The absolute Optimal algorithm requires knowing future requests.

TOP FOLLOW-UPS

- Explain the LRU cache implementation.
- How does the Second Chance algorithm work?

INTERVIEWER TRAP

Don't state that the Optimal algorithm is used in production. It is strictly a theoretical benchmark for comparing other algorithms, as the OS cannot predict future page accesses.

MEMORY TRICK

FIFO = Queue at a bakery. LRU = Checking what tools at the bottom of your drawer haven't been touched recently.

MEMORY MANAGEMENT

Yield: ★★★★★

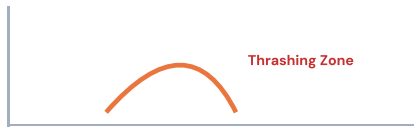
08 – Page Fault & Thrashing

Preventing system degradation caused by continuous paging.

THRASHING DEFINITION

Thrashing occurs when a process spends more time swapping pages in and out of disk than executing instructions. CPU utilization drops to near zero.

CPU Utilization vs Degree of Multiprogramming



WHERE USED IN INDUSTRY

Kubernetes monitors page fault rates of container pods; high thrashing rates trigger automatic horizontal scaling or pod restarts.

INTERVIEW DEPTH

- L1: Page fault handling sequence from trap to disk fetch.
- L2: Thrashing triggers: CPU utilization drop vs page fault rise.
- L3: Locality model of process memory execution paths.
- L4: Page Fault Frequency (PFF) dynamic frame allocation.
- L5: Swap space storage configuration on NVMe vs SSD.

CORE INTERVIEW QUESTION

"Why does the OS scheduler increase multiprogramming when thrashing begins, and how does this make things worse?"

KEYWORDS TO DROP

CPU UTILIZATION DROP

PAGE FAULT STORM

WORKING SET MODEL

LOCALITY MODEL

MODEL ANSWER

"When thrashing starts, processes wait on disk I/O, causing CPU utilization to drop. The OS scheduler mistakes this for low load and launches new processes. These new processes steal pages from existing ones, causing more page faults and worsening thrashing."

DEEP DIVE FOLLOW-UP

→ **"How can the OS mitigate thrashing?"**

By implementing the Working Set Model: estimating how many pages a process needs in its active locality, and suspending processes if total demand exceeds available frames.

REAL INTERVIEW SCENARIO

Interviewer: "How do you detect and fix thrashing in a system?"

Candidate: "We check if CPU utilization drops while paging rate spikes. We fix it by terminating active processes, adding RAM, or using the working-set model to allocate sufficient pages per process."

COMMON MISTAKE

Thinking adding more processes stops thrashing. It worsens it.

TOP FOLLOW-UPS

- What is the working set model?
- How does a page fault rate monitor prevent thrashing?

INTERVIEWER TRAP

Don't confuse context-switch delays with thrashing. Context switching is CPU bound; thrashing is disk I/O bound.

MEMORY TRICK

Thrashing is trying to study 10 books at once when your desk only fits 2, spending all your time opening and closing books.

PROCESS SYNCHRONIZATION

Yield: ★★★★★

09 – Critical Section Problem

Regulating access to shared resources to prevent data corruption.

DEFINITION & REQUIREMENTS

A Critical Section is a code segment accessing shared variables or resources that must not be accessed concurrently by multiple threads.

3 Strict Requirements for any Solution:

1. **Mutual Exclusion:** Only one thread inside critical section at a time.
2. **Progress:** Only threads wanting to enter determine who enters next.
3. **Bounded Waiting:** A thread cannot wait indefinitely to enter.

WHERE USED IN INDUSTRY

Distributed lock managers (like Redis-based Redlock or ZooKeeper) enforce critical section locks across microservices to prevent double-spending bugs.

INTERVIEW DEPTH

- L1: Critical Section problem: Mutual Exclusion, Progress, Bounded Waiting.
- L2: Software critical section guards (Peterson's solution).
- L3: Hardware-assisted synchronization (Test-And-Set, Compare-And-Swap).
- L4: Spinlocks busy-waiting vs blocking locks sleep contexts.
- L5: Lock-free data structures using atomic CAS primitives.

CORE INTERVIEW QUESTION

"What is a Race Condition, and how does a Critical Section solution prevent it?"

KEYWORDS TO DROP

RACE CONDITION

ATOMIC EXECUTION

MUTUAL EXCLUSION

INTERRUPT ENABLE/DISABLE

MODEL ANSWER

"A Race Condition is a flaw where the system's output depends on the execution sequence of threads (e.g. concurrent balance increments). A Critical Section solution uses synchronization primitives (mutexes, locks) to serialize execution, guaranteeing database operations run atomically."

DEEP DIVE FOLLOW-UP

→ "What is Peterson's Solution?"

A classic software-based solution for two processes. It uses a turn variable and an interested array to guarantee mutual exclusion, progress, and bounded waiting.

CRITICAL SECTION DEPTH

Level 1 (Goal): Prevent multiple threads from executing in critical code simultaneously.

Level 2 (Laws): Satisfy Mutual Exclusion, Progress, and Bounded Waiting.

Level 3 (Software): Analyze Peterson's two-process software algorithm limitations.

Level 4 (Hardware): Detail atomic instructions like TestAndSet or CompareAndSwap.

COMMON MISTAKE

Thinking mutual exclusion alone is enough. Progress is essential.

TOP FOLLOW-UPS

- What are the three requirements for critical section?
- How does TestAndSet instruction work?

INTERVIEWER TRAP

Don't say disabling interrupts is a good modern critical section solution. It crashes multi-core CPUs and blocks system clocks.

MEMORY TRICK

Critical Section is a single-occupancy airplane restroom. The lock on the door is the synchronization primitive.

PROCESS SYNCHRONIZATION

Yield: ★★★★★

10 – Mutex vs Semaphore vs Monitor

Contrasting locks, atomic counters, and high-level language constructs.

PRIMITIVE ARCHITECTURE

Mutex: A locking mechanism. Ownership-based; only the thread that locks it can unlock it. Binary state.

Semaphore: An integer variable accessed via atomic functions: `wait()` (P) and `signal()` (V). No ownership. Can be binary or counting.

Monitor: High-level language encapsulation package (like Java synchronized blocks) handling locking implicitly.

Primitive	Concept	Ownership	Signal Safety
Mutex	Key Lock	Yes	High
Semaphore	Count Tokens	No	Medium
Monitor	Object Package	Yes	Highest

WHERE USED IN INDUSTRY

Node.js cluster workers coordinate shared memory ports using atomic buffers and mutex locks to process incoming TCP requests concurrently.

INTERVIEW DEPTH

- L1: Mutex vs Semaphore locking/signaling behaviors.
- L2: Counting semaphores vs Binary semaphores.
- L3: Priority inversion traps and inheritance protocols.
- L4: Sleep-wake synchronization queues inside kernel.
- L5: Deadlock-free locks using lock timeouts and try-lock.

CORE INTERVIEW QUESTION

"Can a thread release a Mutex locked by another thread? How does a Semaphore differ?"

KEYWORDS TO DROP

LOCK OWNERSHIP

COUNTING RESOURCE

WAIT / SIGNAL (P/V)

PRIORITY INVERSION

MODEL ANSWER

"No, a Mutex enforces ownership; only the thread that acquired the lock can release it. A Semaphore has no ownership; any thread can trigger a `signal()` to increment its value, making it optimal for cross-thread scheduling signals."

DEEP DIVE FOLLOW-UP

→ "What is a Binary Semaphore vs a Mutex?"

A binary semaphore is a semaphore with a value of 0 or 1. Unlike a Mutex, it lacks ownership, has no priority inversion protection, and allows signal calls from any thread.

REAL INTERVIEW SCENARIO

Interviewer: "Can a thread release a Mutex it does not own?"

Candidate: "No. Mutexes have strict ownership; only the thread that locks can unlock. Semaphores are signaling variables without ownership, so any thread can call signal/release to unblock a waiter."

COMMON MISTAKE

Saying Mutex is just a binary semaphore. Mutex has ownership, semaphore does not.

TOP FOLLOW-UPS

- What is priority inversion?
- Differentiate spinlock vs mutex.

INTERVIEWER TRAP

Don't confuse Semaphore wait/signal names. `wait()` (P) decrements. `signal()` (V) increments. Memory tip: V for 'Vroom Vroom' (increase/go).

MEMORY TRICK

Mutex = One key to the restroom. Semaphore = A bowl containing 5 keycards for a parking garage.

PROCESS SYNCHRONIZATION

Yield: ★★★★★

11 – Classic Synchronization Problems

Analyzing Producer–Consumer, Readers–Writers, and Dining Philosophers.

PROBLEM DEFINITIONS

Producer–Consumer: Shared finite buffer. Producer shouldn't add if full; consumer shouldn't pull if empty.

Readers–Writers: Multiple readers allowed, but writers need exclusive access. Prevents data corruption.

Dining Philosophers: Resource allocation deadlock test. 5 thinkers sharing 5 chopsticks (needs 2 to eat).

Dining Philosophers Solution Rules:

- Left chopstick:
 i

- Right chopstick:
 $(i+1) \% 5$

- Break asymmetry: odd numbered pick left first, even pick right first (avoids deadlock).



WHERE USED IN INDUSTRY

Browser rendering engines serialize document reads and concurrent canvas writes using read-write locks to prevent screen tearing artifact bugs.

INTERVIEW DEPTH

- L1: Bounded–Buffer (Producer–Consumer) queue sync.
- L2: Readers–Writers problem priority configurations.
- L3: Dining Philosophers resource lock structures.
- L4: Thread synchronization using monitors and condition variables.
- L5: Message passing IPC vs shared memory synchronizations.

CORE INTERVIEW QUESTION

"Explain the Dining Philosophers problem and how to prevent deadlock without using locks."

KEYWORDS TO DROP

CIRCULAR WAIT

ASYMMETRIC PICKUP

CHOPSTICK SEMAPHORES

RESOURCE HIERARCHY

MODEL ANSWER

"Deadlock occurs when all 5 philosophers pick up their left chopstick simultaneously, waiting forever for the right (Circular Wait). We resolve this by breaking the symmetry: odd philosophers pick up left first, even pick up right first. This breaks the cycle."

DEEP DIVE FOLLOW-UP

→ "What is the Readers–Writers priority problem?"
If readers have priority, new readers block writers indefinitely (Starvation). If writers have priority, readers starve. Solved via fair queuing semaphores.



PROBLEM SYNC RULES

Bounded Buffer: Synchronizes producer and consumer. Uses empty/full counting semaphores to block queue boundaries.

Readers–Writers: Multiple readers allowed, but writers get exclusive access. Starvation occurs if readers stream constantly.

Dining Philosophers: Evaluates resource deadlocks. Solved by asymmetric seating or resource acquisition ordering.



COMMON MISTAKE

Forgetting buffer checks in Producer–Consumer.



INTERVIEWER TRAP

Don't suggest a solution that allows only one philosopher to eat at a time. It blocks progress and is highly inefficient.



TOP FOLLOW-UPS

- How does monitor differ from semaphore?
- Explain the dining philosophers deadlock solution.



MEMORY TRICK

Dining Philosophers is a resource lock simulator. Chopsticks are sharing locks. Philosophers are threads.

DEADLOCKS

Yield: ★★★★★

12 – Deadlock: Four Conditions

Analyzing Coffman's four mandatory conditions for system deadlocks.

COFFMAN CONDITIONS

A deadlock is a state where a set of processes are blocked because each holds a resource and waits for another resource held by someone else.

Four Conditions (Must occur simultaneously):

1. **Mutual Exclusion:** Resources cannot be shared.
2. **Hold and Wait:** Process holds a resource while waiting for another.
3. **No Preemption:** Resources cannot be forcibly taken.
4. **Circular Wait:** P1 waits for P2... who waits for P1.

WHERE USED IN INDUSTRY

Database transaction engines (like InnoDB) impose lock hierarchies (e.g., ordering primary key modifications) to break circular wait conditions.

INTERVIEW DEPTH

- L1: Coffman conditions: Mutual Exclusion, Hold-Wait, No Preemption, Circular Wait.
- L2: Deadlock prevention by breaking hold-and-wait.
- L3: Deadlock prevention by breaking circular wait.
- L4: Resource allocation graph cycle tracing loops.
- L5: Livelock vs Deadlock active thread state execution.

CORE INTERVIEW QUESTION

"How can you eliminate 'Circular Wait' inside an application's database code?"

KEYWORDS TO DROP

RESOURCE ORDERING

ACQUISITION SEQUENCE

TOTAL ORDERING

LOCK HIERARCHY

MODEL ANSWER

"We eliminate Circular Wait by establishing a global Lock Order. If Thread A and Thread B both need Lock 1 and Lock 2, they must acquire them in the same order (Lock 1 first, then Lock 2). This prevents a cycle where Thread A holds Lock 1 and Thread B holds Lock 2."

DEEP DIVE FOLLOW-UP

→ "What is the difference between Starvation and Deadlock?"

Deadlock is a circular block of processes (none can progress). Starvation is a process waiting indefinitely for a resource while others bypass it.

DEADLOCK PREVENTION DEPTH

- Level 1 (Lock):** Explain that deadlock is a circular block of threads waiting on locks.
- Level 2 (Coffman):** Identify Mutual Exclusion, Hold-Wait, No Preemption, and Circular Wait.
- Level 3 (Break):** Prevent deadlock by breaking Circular Wait (imposing resource indices).
- Level 4 (Starve):** Differentiate deadlock (stuck locks) from livelock (active state shifts).

COMMON MISTAKE

Not knowing circular wait is the essential trigger.

TOP FOLLOW-UPS

- Differentiate deadlock vs starvation.
- How do we prevent circular wait?

INTERVIEWER TRAP

Don't say you can resolve deadlocks by ignoring them. The 'Ostrich algorithm' is used in commodity OS designs, but it is not a technical solution.

MEMORY TRICK

Remember the Coffman conditions: Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait (MHNC).

DEADLOCKS

Yield: ★★★★★

13 – Banker's Algorithm

Deadlock avoidance using resource allocation state validation.

SAFE VS UNSAFE STATES

Developed by Dijkstra. Used for deadlock avoidance in multi-resource systems. Evaluates if allocating a resource keeps the system in a **Safe State**.

Matrix Equations:

$$\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$$

Allocation Check Sequence

If $\text{Need} \leq \text{Available}$, the process runs to completion, releases resources, and updates: $\text{Available} = \text{Available} + \text{Allocation}$.

WHERE USED IN INDUSTRY

Safety-critical aviation control systems use Banker's algorithm logic to pre-calculate resource limits before starting autonomous flight routines.

INTERVIEW DEPTH

- L1:** Deadlock avoidance safe vs unsafe state boundaries.
- L2:** Banker's algorithm matrix structures (Allocation, Max, Available, Need).
- L3:** Safety check loop algorithm steps.
- L4:** Resource request algorithm interception logic.
- L5:** Banker's algorithm time complexity limitations.

CORE INTERVIEW QUESTION

"Does an Unsafe State always lead to a Deadlock?"

KEYWORDS TO DROP

SAFE PATH

UNSAFE STATE

WORST-CASE DEMAND

RESOURCE ALLOCATION

MODEL ANSWER

"No, an Unsafe State does not guarantee a deadlock. It only indicates that the system cannot guarantee a safe execution path if all processes request their maximum declared resources simultaneously. If processes yield early, deadlock may not occur."

DEEP DIVE FOLLOW-UP

→ **"Why is the Banker's algorithm not used in general-purpose OS designs?"**

It requires processes to declare their maximum resource needs in advance, which is impossible for dynamic, general-purpose workloads.

REAL INTERVIEW SCENARIO

Interviewer: "Why isn't Banker's algorithm used in general-purpose operating systems?"

Candidate: "It requires processes to declare their maximum resource needs in advance, which dynamic desktop applications cannot do. It also assumes a fixed resource pool and has $O(M \cdot N^2)$ overhead."

COMMON MISTAKE

Thinking unsafe state means deadlock. Unsafe only means deadlock is possible.

TOP FOLLOW-UPS

- What is a safe state?
- What is the time complexity of Banker's?

INTERVIEWER TRAP

Don't confuse Deadlock Avoidance (Banker's) with Deadlock Prevention (breaking Coffman conditions). Prevention is static; avoidance is dynamic.

MEMORY TRICK

Banker's is a loan officer checking if the bank has enough cash to cover worst-case credit lines before approving a loan.

DEADLOCKS

Yield: ★★★★★

14 – Deadlock Detection & Recovery

Identifying active deadlock states and restoring system stability.

DETECTION STRATEGY

If a system doesn't prevent or avoid deadlocks, it must periodically run a detection algorithm (Wait-For Graph search) to find cycles.

Recovery Actions:

1. **Process Termination:** Abort all deadlocked processes, or abort them one-by-one until the cycle is broken.
2. **Resource Preemption:** Reclaim resources from one process and assign to others (causes rollback issues).

WHERE USED IN INDUSTRY

PostgreSQL runs background deadlock detection workers to trace wait-for-graphs and abort blocked transactions to resume database execution.

INTERVIEW DEPTH

- L1: Deadlock detection algorithms for single/multiple resource types.
- L2: Wait-for-graph (WFG) reduction loops.
- L3: Recovery strategies: process termination vs resource preemption.
- L4: Rollback points to recover database state post deadlock.
- L5: Selecting victim processes based on CPU usage and lock durations.

CORE INTERVIEW QUESTION

"How does a Wait-For Graph (WFG) identify deadlocks, and how does it differ from a Resource Allocation Graph (RAG)?"

KEYWORDS TO DROP

CYCLE DETECTION

DIRECTED GRAPH

NODE SIMPLIFICATION

CYCLE SEARCH

MODEL ANSWER

"A Resource Allocation Graph (RAG) contains nodes for both processes and resource types. A Wait-For Graph is a simplified version containing only process nodes. If there is a cycle in a Wait-For Graph, a deadlock exists (for single-unit resource systems)."

DEEP DIVE FOLLOW-UP

→ "What is rollback in deadlock recovery?"

Saving checkpoint states of processes. During recovery, the OS rolls a process back to a safe checkpoint state, relinquishing its locks.

DETECTION & RECOVERY DEPTH

- Level 1 (Check):** Run detection algorithms periodically using Resource Allocation Graphs.
- Level 2 (Cycle):** Detect cycles in single-instance systems using Wait-For Graph walks.
- Level 3 (Kill):** Recover by terminating deadlock cycle processes one-by-one.
- Level 4 (Rollback):** Preempt resources, rolling processes back to saved safe checkpoints.

COMMON MISTAKE

Thinking deadlock detection is overhead-free. It adds CPU cycles.

TOP FOLLOW-UPS

- How often should a deadlock detection run?
- What is resource preemption?

INTERVIEWER TRAP

Don't suggest rebooting the server as the primary deadlock recovery mechanism. It causes data loss; graceful process termination is preferred.

MEMORY TRICK

RAG has shape nodes for resources. WFG simplifies them, leaving only arrows connecting blocked threads.

FILE SYSTEMS & STORAGE

Yield: ★★★★★

15 – File Systems & Inodes

How UNIX-like operating systems structure files, directories, and metadata.

THE INDEX NODE (INODE)

An inode is a kernel structure containing metadata about a file. Crucially, **the inode does NOT store the file name** (names are stored in directory tables).

Inode Metadata Fields:

- File Size (Bytes)
- Owner (UID/GID) & Permissions
- Modification Timestamp
- 12 Direct Pointers to data blocks
- Indirect, Double Indirect, Triple Indirect Pointers

WHERE USED IN INDUSTRY

Modern cloud filesystems (like AWS EFS or GCP Filestore) manage metadata pointers using inodes, optimizing index block lookups for mass scale.

INTERVIEW DEPTH

- L1:** File allocation schemes: Contiguous, Linked, Indexed.
- L2:** Inode structure: Direct blocks, single/double/triple indirect blocks.
- L3:** Directory structure mapping filenames to inode numbers.
- L4:** File Allocation Table (FAT) vs Unix Fast File System (FFS) tables.
- L5:** Journaling filesystems transaction logs write-ahead logs.

CORE INTERVIEW QUESTION

"Explain the difference between Hard Links and Soft Links (Symbolic Links) in UNIX."

KEYWORDS TO DROP

INODE REFERENCE COUNT

PATH POINTER

DIRECTORY ENTRY

LINK INVALIDATION

MODEL ANSWER

"A Hard Link creates a new directory entry pointing to the same underlying inode address; deleting the original file doesn't break the hard link. A Soft Link is a separate pointer file containing the file path string; if the original is deleted, the soft link becomes broken."

DEEP DIVE FOLLOW-UP

→ **"What is the maximum file size limit of an inode?"**
Determined by block size and indexing depth. With a 4KB block size, 12 direct + single/double/triple indirect pointers support file sizes up to 4TB.

INODE STORAGE DEPTH

Level 1 (Metadata): Inode stores file size, permissions, owner, and block pointers.

Level 2 (Block Pointers): Direct pointers link to file blocks; indirect pointers link to block lists.

Level 3 (Pointers): Single, double, and triple indirect pointers enable large file limits.

Level 4 (Links): Hard links point to the same inode; symbolic links point to paths.

COMMON MISTAKE

Saying FAT has no index table. The File Allocation Table is a linked allocation index.

TOP FOLLOW-UPS

- What is a superblock?
- How do hard links differ from soft links?

INTERVIEWER TRAP

Don't confuse file content changes with link changes. Both hard and soft link changes modify the same underlying file content.

MEMORY TRICK

Hard Link = A second name tag pointing to the same person. Soft Link = An address card with a map to their house.

FILE SYSTEMS & STORAGE

Yield: ★★★★★

16 – Disk Scheduling Algorithms

Minimizing head seek time in magnetic hard drives.

ALGORITHM VARIATIONS

FCFS: Fair but slow; causes random head jumps.**SSTF:** Shortest Seek Time First. Starves distant requests.**SCAN:** Elevator algorithm. Moves head end-to-end, serving requests along the way.**LOOK:** Like SCAN, but turns around at the last request instead of going to the physical disk edge.

SCAN vs C-SCAN (Circular SCAN)

SCAN: Serves incoming and outgoing sweeps.**C-SCAN:** Serves only in one direction, returning instantly to the start for predictability.

WHERE USED IN INDUSTRY

Solid-State Drive (SSD) controllers use internal Flash Translation Layers (FTL) to route flash sweeps, rendering traditional elevator disk sweeps obsolete.

INTERVIEW DEPTH

- L1:** Disk arm scheduling algorithms: FCFS, SSTF, SCAN, C-SCAN.
- L2:** SSTF starvation risks for boundary cylinder sectors.
- L3:** SCAN/C-SCAN boundary bounce cycles.
- L4:** Elevator algorithm execution in magnetic disk drivers.
- L5:** SSD flash block writes garbage collection loops.

CORE INTERVIEW QUESTION

"Why does SSTF suffer from starvation, and how does SCAN solve it?"

KEYWORDS TO DROP

SHORTEST SEEK TIME

LOCALITY OF REQUESTS

ELEVATOR SWEEP

FAIR QUEUEING

MODEL ANSWER

"SSTF services the request closest to the current head position. If new requests keep arriving near the head, distant requests starve. SCAN sweeps end-to-end like an elevator, guaranteeing all queued sectors are serviced in a single pass."

DEEP DIVE FOLLOW-UP

→ **"Do SSDs need disk scheduling?"**

No. SSDs have no moving physical parts (no seek time). SSD controllers use direct lookup tables, making disk scheduling obsolete for solid-state storage.



DISK SCAN TRADE-OFFS

SSTF: Serviced closest request. Minimizes seek time but suffers from severe starvation.

SCAN (Elevator): Head moves in one direction to end, then reverses. Solves starvation but causes queue unfairness.

C-SCAN (Circular): Head sweeps in one direction, then returns to beginning without servicing. Ensures uniform wait time.

⚠ COMMON MISTAKE

SSTF is starvation-free. It is not; continuous local requests starve distant ones.

🔄 TOP FOLLOW-UPS

- Why does SSTF cause starvation?
- What is the difference between SCAN and C-SCAN?

🔴 INTERVIEWER TRAP

Don't confuse C-SCAN with C-LOOK. C-SCAN goes to the absolute physical edge of the disk. C-LOOK turns around at the last queued request in that direction.

💡 MEMORY TRICK

SCAN is a city bus completing its route. SSTF is a taxi choosing the closest fare next.

OPERATING SYSTEM ARCHITECTURE

Yield: ★★★★★

17 – Monolithic vs Microkernel

Analyzing kernel size and extension patterns.

CORE ARCHITECTURES

Monolithic Kernel: All OS services (scheduler, file system, drivers) run in kernel space. Fast execution, but a crash in a driver crashes the OS (e.g. Linux).

Microkernel: Only essential services (IPC, memory) run in kernel space; others run in user space. Stable, but slow due to IPC overhead (e.g. Mach).

Monolithic

Drivers in Kernel
Fast calls
High crash risk

Microkernel

Drivers in User space
IPC overhead
High isolation

WHERE USED IN INDUSTRY

Edge routers use monolithic kernels (like Linux) for maximum packet-routing speed, while secure smart cards use microkernels to isolate cryptographic tasks.

INTERVIEW DEPTH

- L1: Monolithic vs Microkernel architecture structures.
- L2: IPC communication overhead inside Microkernels.
- L3: Hybrid kernel architectures (Windows NT kernel).
- L4: Exokernel bare-metal resource multiplexing.
- L5: Loadable Kernel Modules (LKMs) dynamic load pathways.

CORE INTERVIEW QUESTION

"Why does a Microkernel suffer from performance overhead compared to a Monolithic kernel?"

KEYWORDS TO DROP

MESSAGE PASSING (IPC)

CONTEXT SWITCH COUNT

USER/KERNEL BOUNDARY

INDIRECT CALLS

MODEL ANSWER

"In a Monolithic kernel, OS services communicate via direct function calls. In a Microkernel, services run in user space. Any request (e.g., writing a file) requires message passing (IPC), causing multiple context switches between user and kernel space, degrading performance."

DEEP DIVE FOLLOW-UP

→ "What is a Hybrid Kernel?"

A compromise (like Windows NT or macOS Darwin) that runs core services inside kernel space for speed, but adopts modular microkernel layouts.

REAL INTERVIEW SCENARIO

Interviewer: "Why does macOS use a hybrid kernel rather than a pure microkernel?"

Candidate: "Pure microkernels require extensive context switches and IPC message passing for simple tasks, degrading performance. Hybrid kernels keep security benefits but run core services in kernel space."

COMMON MISTAKE

Saying Windows is strictly microkernel. It is a hybrid kernel.

TOP FOLLOW-UPS

- What is monolithic architecture?
- What is microkernel IPC overhead?

INTERVIEWER TRAP

Don't state that Linux is unstable. While monolithic, modular code architecture and testing make Linux robust in production systems.

MEMORY TRICK

Monolithic is an open-plan office (fast sharing, one sick person infects all). Microkernel is individual cubicles.

OPERATING SYSTEM ARCHITECTURE

Yield: ★★★★★

18 – System Calls (fork, exec)

Programmatic interface to kernel space and process spawning.

SYSTEM CALL INTERFACE

A system call is the programmatic method by which a user-space program requests a service from the kernel. Uses software interrupts (traps).

Key Calls:

fork(): Spawns a child process. Returns PID of child to parent, and 0 to the child.

exec(): Replaces current process image with a new executable program.

WHERE USED IN INDUSTRY

Docker containers restrict system calls (like `ptrace`` or `sys_chroot``) using SECCOMP profiles to prevent privilege escalation container escape bugs.

INTERVIEW DEPTH

- L1:** System call interrupt trap instruction execution.
- L2:** User space to kernel space hardware CPU ring transitions.
- L3:** System call table lookup dispatch sequence.
- L4:** Parameter passing mechanisms (CPU registers vs stack memory).
- L5:** Virtual System Calls (vDSO) to avoid user-kernel switch context costs.

CORE INTERVIEW QUESTION

"Explain the difference in execution between `fork()` and `exec()`. How do they work together?"

KEYWORDS TO DROP

DUPLICATE PROCESS

OVERWRITE ADDRESS SPACE

RETURN VALUE CHECK

COPY-ON-WRITE (COW)

MODEL ANSWER

"`fork()` clones the calling process, creating a duplicate child with a matching memory layout. `exec()` replaces the current process's address space with a new program image. To run a new program, a process calls `fork()` first, then the child immediately calls `exec()`."

DEEP DIVE FOLLOW-UP

→ "What is Copy-on-Write (COW) in `fork()`?"

A optimization where child and parent share the same physical memory frames initially. Physical frames are duplicated only when parent or child writes to them.

SYSTEM CALL DEPTH

Level 1 (System): Syscalls provide the user-to-kernel interface for file/process operations.

Level 2 (Trap): CPU switches mode using a trap instruction, passing parameters via registers.

Level 3 (Interrupt): Kernel queries the Interrupt Vector Table to find the handler.

Level 4 (ISR): Interrupt Service Routine executes, resetting the CPU privilege state.

COMMON MISTAKE

Confusing library calls with syscalls (e.g. `printf`` calls `write`` syscall).

TOP FOLLOW-UPS

- Differentiate hardware vs software interrupts.
- What is the purpose of interrupt vector table?

INTERVIEWER TRAP

Remember that `fork()` duplicates the process at the exact line of execution. Any code below the `fork()` statement is executed by BOTH processes.

MEMORY TRICK

`fork()` = Cloning yourself. `exec()` = Putting on a mask to become someone else.

19 – Inter-Process Communication

Mechanisms for processes to share data and coordinate activities.

IPC MODELS

Processes run in isolated address spaces. Sharing data requires explicit IPC channels managed by the operating system kernel.

Shared Memory: Processes share a physical memory segment. Fastest, but requires synchronization (mutexes) to prevent race conditions.

Message Passing: Processes send messages via kernel message queues. System-call bound, but safe across networks.

WHERE USED IN INDUSTRY

Microservice communication protocols (like gRPC or HTTP/2) run internal IPC pipes (like Unix domain sockets) when running on the same host to bypass TCP overhead.

INTERVIEW DEPTH

- L1: IPC mechanisms: Shared Memory, Message Queues, Pipes, Sockets.
- L2: Anonymous pipes vs Named pipes (FIFOs) folder files.
- L3: Shared memory page mapping context setup.
- L4: Blocking vs non-blocking message queue reads.
- L5: Unix domain socket performance vs network TCP sockets.

CORE INTERVIEW QUESTION

"Compare Shared Memory and Message Passing in IPC. When is each preferred?"

KEYWORDS TO DROP

ADDRESS SPACE SHARING

KERNEL OVERHEAD

SOCKET COMMUNICATION

RACE CONDITION RISK

MODEL ANSWER

"Shared Memory is preferred for high-speed, local data sharing (no kernel intervention after setup). However, it requires software synchronization. Message Passing is preferred for smaller payloads or distributed systems, as the kernel handles message buffers safely."

DEEP DIVE FOLLOW-UP

→ **"What is a Named Pipe (FIFO) in UNIX?"**

An extension of standard anonymous pipes. Named pipes appear as special files on disk, allowing unrelated processes to communicate.

SHARED MEMORY VS MESSAGE PASSING

Shared Memory: High-speed mapping of RAM between process spaces. Zero kernel overhead after setup, but requires process-level synchronization (mutexes).

Message Passing: Kernel-mediated buffers (pipes/queues). Slower due to syscall overhead, but safe and easy to implement.

COMMON MISTAKE

Shared memory requires system calls for every write. Syscall is only needed to map it.

TOP FOLLOW-UPS

- Explain pipe vs socket.
- Differentiate synchronous vs asynchronous message passing.

INTERVIEWER TRAP

Don't say shared memory has no race conditions. It is vulnerable to race conditions if developers fail to implement locks.

MEMORY TRICK

Shared Memory = Two programmers editing the same Google Doc.
Message Passing = Sending emails back and forth.

MEMORY MANAGEMENT

Yield: ★★★★★

20 – Fragmentation (Internal vs External)

Wasted space within allocated blocks vs unallocated holes.

WASTED SPACE TAXONOMY

Internal Fragmentation: Memory block allocated to a process is larger than requested. The unused space inside the block is wasted (occurs in fixed-partition paging).

External Fragmentation: Total free space exists to satisfy a request, but it is not contiguous (occurs in variable-partition segmentation).

External Fragmentation

[Proc A: 4MB] [Free: 2MB] [Proc B: 4MB] [Free: 2MB]

Cannot satisfy a new request of 3MB contiguously!

CORE INTERVIEW QUESTION

"How does Memory Compaction resolve external fragmentation? What is its main limitation?"

KEYWORDS TO DROP

MEMORY COMPACTION

RELOCATION REGISTER

EXECUTION-TIME BINDING

I/O PERFORMANCE DROP

MODEL ANSWER

"Compaction sweeps memory to slide all allocated blocks to one end, leaving a single large free block. Its main limitation is computational overhead: moving megabytes of active process memory is slow, requiring suspension of running programs."

DEEP DIVE FOLLOW-UP

→ "What allocation strategies choose free blocks?"

First-Fit: Allocates the first hole that fits. **Best-Fit:** Allocates the smallest hole that fits (worsens external fragmentation). **Worst-Fit:** Allocates the largest hole.

REAL INTERVIEW SCENARIO

Interviewer: "Why does segmentation lead to external fragmentation?"

Candidate: "Segments are logical units of variable size. As segments are allocated and deallocated from memory, the remaining free spaces become chopped into small, isolated holes that cannot satisfy large segment requests."

WHERE USED IN INDUSTRY

Intel hardware memory controllers support segmentation registers to enforce sandbox boundaries for browser javascript sandboxing environments.

INTERVIEW DEPTH

- L1: Logical segmentation vs physical paging structures.
- L2: External fragmentation causing alloc failures.
- L3: Segmentation registers base/limit offset checks.
- L4: Paged segmentation address translation maps.
- L5: Dynamic memory allocator heap fragmentation compaction.

COMMON MISTAKE

Saying segmentation uses fixed size. Segments have dynamic lengths.

TOP FOLLOW-UPS

- What is compaction in memory?
- Differentiate internal vs external fragmentation.

INTERVIEWER TRAP

Don't say best-fit is always optimal. Best-fit produces tiny, unusable leftover holes, worsening external fragmentation over time.

MEMORY TRICK

Internal = Buying a size 10 shoe for a size 9 foot. External = Having 5 empty single seats scattered across a bus, but a group of 3 cannot sit together.

MEMORY MANAGEMENT

Yield: ★★★★★

21 – TLB (Translation Lookaside Buffer)

Accelerating address translations using associative cache hardware.

TRANSLATION ACCELERATION

Every memory access in paging requires two RAM lookups: one to read the page table, and one to read the actual data. This halves CPU performance.

TLB: A small, ultra-fast hardware cache built into the MMU containing recently resolved page-to-frame mappings.

Virtual Page Number
↓ check TLB cache

TLB Hit: Returns Frame ID instantly (1 clock cycle)

TLB Miss: Resolves via Page Table in RAM (100 clock cycles)

WHERE USED IN INDUSTRY

High-frequency trading architectures run on CPU cores configured with HugePages (2MB or 1GB frames) to maximize TLB hits and reduce translation latency.

INTERVIEW DEPTH

- L1:** Translation Lookaside Buffer translation caching.
- L2:** TLB hit ratio calculation and access time lookup.
- L3:** TLB miss page table search context walks.
- L4:** TLB flushes during context switches vs ASID tags.
- L5:** Multi-level page table TLB access penalties.

CORE INTERVIEW QUESTION

"Explain the concept of a TLB Hit and a TLB Miss. How does it affect effective access time?"

KEYWORDS TO DROP

EFFECTIVE MEMORY ACCESS

MMU ASSOCIATIVE CACHE

HARDWARE PAGE WALK

CONTEXT SWITCH FLUSH

MODEL ANSWER

"On a TLB Hit, the physical address is resolved in 1 clock cycle. On a TLB Miss, the CPU must perform a 'page walk' in RAM to resolve the frame, taking 100+ clock cycles. High TLB hit rates (typically >99%) keep effective memory access times low."

DEEP DIVE FOLLOW-UP

→ "What is a TLB flush?"

During a context switch, the OS flushes the TLB cache because the new process has a completely different page-to-frame mapping layout.

TLB OPERATION DEPTH

Level 1 (Cache): TLB is a fast hardware cache inside MMU to store active translations.

Level 2 (Lookup): On miss, CPU performs multi-level page table walks in RAM.

Level 3 (Context): Context switch flushes TLB to prevent cross-process data leakage.

Level 4 (Tags): Address Space Identifiers (ASID) tag translations, avoiding flush costs.

COMMON MISTAKE

TLB caches program data. It only caches address translations.

TOP FOLLOW-UPS

- What is TLB reach?
- Explain TLB miss handling.

INTERVIEWER TRAP

Don't confuse TLB with CPU L1/L2 caches. TLB caches virtual address translations; L1/L2 caches actual memory data bytes.

MEMORY TRICK

TLB is looking up a phone number in your contacts list instead of walking to the municipal library directory.

OPERATING SYSTEM SECURITY

Yield: ★★★★★

22 – User Mode vs Kernel Mode

Securing kernel space via dual-mode CPU privilege execution.

PRIVILEGED RINGS

Modern CPUs enforce privilege separation to prevent user applications from crashing the system. Controlled by a **Mode Bit** in the CPU status register.

User Mode (Mode Bit = 1): Restricted instructions. Direct hardware access blocked.

Kernel Mode (Mode Bit = 0): Full access to physical hardware and kernel memory spaces.

WHERE USED IN INDUSTRY

Nvidia GPU driver modules transition between user space APIs (like CUDA) and privileged kernel modules to coordinate parallel pipeline runs.

INTERVIEW DEPTH

- L1:** Privileged ring-0 kernel mode vs ring-3 user mode.
- L2:** Kernel mode instruction protection checks.
- L3:** Privilege escalation attacks using stack overflow buffer writes.
- L4:** Hardware interrupt vector table routing.
- L5:** Context save structures (trap frames) in kernel stack.

CORE INTERVIEW QUESTION

"What triggers a transition from User Mode to Kernel Mode?"

KEYWORDS TO DROP

PRIVILEGED INSTRUCTION

SOFTWARE INTERRUPT (TRAP)

HARDWARE EXCEPTION

SYSTEM CALL TRAP

MODEL ANSWER

"Transition is triggered by three events: 1. System Calls (programs requesting file writes via software traps). 2. Hardware Interrupts (keyboard inputs or NIC signals). 3. Hardware Exceptions (Zero Division or Page Faults). The CPU sets the mode bit to 0 and jumps to the interrupt vector table."

DEEP DIVE FOLLOW-UP

→ **"What is a privileged instruction?"**

Instructions that can only be executed in kernel mode, such as clearing RAM, disabling hardware interrupts, or writing to status registers.

REAL INTERVIEW SCENARIO

Interviewer: "What happens if a user-mode application tries to run a privileged instruction?"

Candidate: "The CPU hardware blocks it, raises a general protection fault interrupt, traps to the kernel handler, and terminates the user process with a segmentation fault."

COMMON MISTAKE

User apps can write directly to disk. They must trap to kernel mode.

TOP FOLLOW-UPS

- What is dual-mode operation?
- List some privileged instructions.

INTERVIEWER TRAP

Don't assume running programs as 'root' or 'Administrator' elevates them to kernel mode. Root programs still run in User Mode, but have file permission privileges.

MEMORY TRICK

User Mode is a hotel guest (allowed in room and lobby). Kernel Mode is hotel staff with master keycards.

OPERATING SYSTEM EXECUTION

Yield: ★★★★★

23 – Dynamic vs Static Linking

Combining code libraries at compile time vs load time.

BINARY COMPILATION

Linking resolves references to external libraries (such as ``libc`` or ``stdio``).

Static Linking: Library code is copied directly into the final executable. Creates a large self-contained binary.

Dynamic Linking: Shared libraries (``so`` or ``dll`` files) are loaded into RAM once, and referenced by multiple programs at runtime.

Static (.a / .lib)

Self-contained binary
Large file size
No runtime dependency

Dynamic (.so / .dll)

Shared in RAM
Tiny file size
Update lib without compile



WHERE USED IN INDUSTRY

Application runtimes (like Java Virtual Machine or Node) resolve dynamic library bindings at startup, lazy-loading shared object files as needed.

INTERVIEW DEPTH

- L1:** Static linking object merges vs Dynamic linking stub calls.
- L2:** Dynamic Linker runtime relocations.
- L3:** Position Independent Code (PIC) global offset tables.
- L4:** Loader heap setup and environment variables parser.
- L5:** Lazy binding PLT/GOT resolution pathways.

CORE INTERVIEW QUESTION

"Why is dynamic linking preferred in modern operating systems?"

KEYWORDS TO DROP

SHARED MEMORY SEGMENT

DISK SPACE ECONOMY

DYNAMIC LINKER (LD.SO)

DLL HELL VERSIONING

MODEL ANSWER

"Dynamic linking saves disk space and RAM by loading shared libraries once into memory. It also allows updating libraries (e.g. patching security bugs) without recompiling the applications that depend on them."

DEEP DIVE FOLLOW-UP

→ "What is a DLL stub?"

A small placeholder compiled into static code indicating how to locate and load the dynamic library at runtime.



STATIC VS DYNAMIC LINKING

Static Linking: Library code is copied directly into the binary at compile time. Large executables, but runs independently without runtime dependencies.

Dynamic Linking: Binary contains pointers to shared libraries (`.so/dll`). Saves disk/RAM space, but fails if libraries are missing.

COMMON MISTAKE

Static linking is runtime loadable. It is embedded at compile time.

TOP FOLLOW-UPS

- What does a linker do?
- Differentiate DLL vs static library.

INTERVIEWER TRAP

Don't assume dynamic linking is always better. Static binaries are highly portable, making them ideal for containerized microservices (Docker).

MEMORY TRICK

Static = Copying a recipe into your cookbook. Dynamic = Linking to a YouTube cooking tutorial.

OPERATING SYSTEM OPERATIONS

Yield: ★★★★★

24 – Spooling vs Buffering

Managing speed disparities between CPU and I/O devices.

I/O INTERMEDIARIES

Spooling (Simultaneous Peripheral Operations On-Line):

Buffers I/O operations on disk storage. Acts as a queue for slow, non-shareable devices (e.g. print queues).

Buffering: Stores data in RAM temporarily while being transferred between fast CPU registers and slow I/O blocks (e.g. video pre-loading).

WHERE USED IN INDUSTRY

Enterprise network print systems and queuing services (like RabbitMQ) spool batch requests onto persistent queues to decouple fast clients from slow devices.

INTERVIEW DEPTH

- L1: Spooling concurrent batch queues vs Buffering streams.
- L2: Spooling temporary storage directories.
- L3: Printer daemon spool files parsing sweeps.
- L4: Message broker disk spools and consumer offsets.
- L5: Spool overflow crash mitigation configurations.

CORE INTERVIEW QUESTION

"Contrast Spooling and Buffering. How do they differ in storage media and target devices?"

KEYWORDS TO DROP

DISK QUEUE SPOOL

RAM MEMORY BUFFER

PRINTER SPOOLER

TEMPORARY STORAGE

MODEL ANSWER

"Spooling uses disk storage to queue operations for slow, non-shareable devices, allowing the CPU to hand off tasks and continue. Buffering uses RAM to smooth out speed mismatches between active channels during a transfer."

DEEP DIVE FOLLOW-UP

→ **"Why do video players use buffering?"**

Network download speeds fluctuate. Buffering pre-downloads video frames into RAM to ensure smooth playback during network dips.

QUEUE & SPOOL DEPTH

Level 1 (Concept): Spooling uses disk queues to coordinate slow I/O device access.

Level 2 (Spool): Contrast buffering (RAM buffer for speed matches) with spooling (disk queues).

Level 3 (Concur): Spooling allows overlapping I/O of multiple concurrent jobs.

Level 4 (Kernel): Print spooler process reads disk queue blocks and sends them to printer NIC.

COMMON MISTAKE

Spooling is identical to buffering. Buffering overlaps I/O of single jobs; spooling queues multiple jobs.

TOP FOLLOW-UPS

- Why is spooling called concurrent?
- Where is the spool buffer located?

INTERVIEWER TRAP

Don't say spooling and buffering are identical. Spooling is disk-based and handles queuing; buffering is RAM-based and handles temporary speed smoothing.

MEMORY TRICK

Spooling = A mailbox holding packages until you collect them.
Buffering = Holding a sip of hot coffee in your mouth before swallowing.

OPERATING SYSTEM ARCHITECTURES

Yield: ★★★★★

25 – RTOS vs GPOS

Deterministic timing constraints vs fair throughput balancing.

SYSTEM PRIORITIES

RTOS (Real-Time OS): Designed for embedded systems requiring microsecond precision. Focuses on ****determinism**** (predictable deadlines).

GPOS (General Purpose OS): Designed for general desktops and servers. Focuses on ****throughput**** and fair scheduling (e.g. Windows, macOS).

Metric	RTOS	GPOS
Goal	Meet Deadlines	Fair Resource Sharing
Latency	Low and Predictable	High and Variable
Scheduling	Priority Preemptive	Fair Share / Time Sliced
Examples	VxWorks, FreeRTOS	Linux, macOS, Windows



WHERE USED IN INDUSTRY

Autonomous vehicle control units use real-time kernels (like QNX or FreeRTOS) to guarantee execution of collision avoidance logic within milliseconds.

INTERVIEW DEPTH

- L1:** Hard vs Soft RTOS deadlines guarantees.
- L2:** Rate Monotonic Scheduling (RMS) static priority math.
- L3:** Earliest Deadline First (EDF) dynamic scheduling limits.
- L4:** Interrupt latency and dispatch latency limits.
- L5:** Priority ceiling protocol to mitigate priority inversion.

CORE INTERVIEW QUESTION

"What is the difference between a Hard RTOS and a Soft RTOS?"

KEYWORDS TO DROP

DETERMINISTIC BOUNDS

MISSED DEADLINE CRASH

PREDICTABLE LATENCY

EMBEDDED CONTROL

MODEL ANSWER

"A Hard RTOS guarantees that critical tasks complete within strict deadlines; missing a deadline is treated as a total system failure (e.g., pacemaker or ABS brakes). A Soft RTOS treats deadlines as goals, accepting occasional delays (e.g., video player)."

DEEP DIVE FOLLOW-UP

→ **"What is priority inversion in RTOS?"**

A bug where a low-priority task holds a resource needed by a high-priority task, while a medium-priority task preempts the low-priority one, delaying the high-priority task.

REAL INTERVIEW SCENARIO

Interviewer: "Why is standard Linux not suitable for safety-critical real-time applications?"

Candidate: "Standard Linux optimizes for average throughput, introducing unpredictable scheduling latency and priority inversion. Safety-critical systems require an RTOS to guarantee hard deadline constraints."

COMMON MISTAKE

Thinking RTOS is faster at math than general OS. It is just more predictable.

TOP FOLLOW-UPS

- Differentiate hard vs soft RTOS.
- What is rate monotonic scheduling?

INTERVIEWER TRAP

Don't assume RTOS is faster at math than GPOS. It is simply more predictable; it guarantees a response within a strict timeframe, even if slow.

MEMORY TRICK

GPOS = A bus driver driving fast but stopping for passengers. RTOS = An ambulance driving on a planned route with priority.

OS Last-Minute Revision Sheet

Top Algorithms, Deadlock Laws, and High-Yield Summaries

⚡ SCHEDULING & MEMORY SUMMARY

SJF	Optimal CPU scheduling, minimizes wait times.
Round Robin	Uses time-slicing (quantum), ensures fairness.
Paging	Virtual page → Physical frame map (No external frag).
Segmentation	Variable-length blocks (stack/heap, causes external frag).
Belady's Anomaly	FIFO page fault increases as physical frames increase.

🔧 OPERATING SYSTEM LAWS

- **Coffman Laws:** Mutual exclusion, hold-wait, no-preemption, circular wait.
- **Banker's Goal:** Allocate resource ONLY if it leaves system in Safe State.
- **TLB Mission:** Cache page table translations to speed paging.
- **Aging Solution:** Solve scheduling starvation by incrementing wait priorities.
- **Thrashing Trigger:** CPU spends more time swapping pages than running tasks.

💡 TOP 5 INTERVIEW CONCEPTS TO RECALL

1. **Mutex vs Semaphore:** Mutex = locking mechanism with ownership (thread-owned). Semaphore = signaling tool (counting).
2. **Zombie vs Orphan:** Zombie = exited but parent hasn't wait()-ed. Orphan = active but parent died, init adopts it.
3. **Context Switch Cost:** CPU registers are swapped out to kernel, cache is flushed. Significant kernel CPU overhead.
4. **Peterson's Rule:** Software critical-section guard. Satisfies mutual exclusion, progress, and bounded wait.
5. **Micro vs Monolithic:** Monolithic runs all inside kernel space (fast). Micro runs in user space, communicating via slower IPC.

🎯 QUICK SELF-TEST CHECKLIST

☐ Differentiate process and thread ☐ List Coffman deadlock conditions ☐ Trace LRU page hits

OS Comparison Cheat Sheet

Quick Reference Contrast Tables for Fresher Interviews

PROCESS VS THREAD

Feature	Process	Thread
Address Space	Independent, private	Shared with parent
Overhead	High (heavy creation)	Low (lightweight)
Switching Cost	High (flushes TLB)	Low (preserves TLB)
Fault Isolation	Strong (crashes alone)	Weak (crashes process)

PAGING VS SEGMENTATION

Feature	Paging	Segmentation
Block Size	Fixed (e.g., 4 KB)	Variable (logical)
Fragmentation	Internal (last page)	External (holes)
User Visibility	Invisible (OS/Hardware)	Visible (Programmer)
Address Format	Page # + Offset	Segment # + Offset

MUTEX VS SEMAPHORE

Feature	Mutex	Semaphore
Mechanism	Locking (Strict Excl)	Signaling variable
Ownership	Yes (Only owner unlocks)	No (Any thread signals)
Types	Single binary lock	Binary and Counting
Value	0 (Locked) or 1 (Free)	Non-negative integer

DEADLOCK VS STARVATION

Feature	Deadlock	Starvation
Description	Circular block state	Indefinite wait state
Process Status	Blocked (waiting)	Ready (not scheduled)
System Action	Requires intervention	Requires Aging priority
Trigger	Mutual lock request	Biased scheduler rules

USER MODE VS KERNEL MODE

Feature	User Mode	Kernel Mode
Privileges	Restricted instructions	Full hardware access
Crash Impact	Process is terminated	System crashes (Panic)
Address Space	User space mappings	Kernel space direct
Transitions	Traps / System Calls	Resets state flag

Top Expected Interview Questions

Placement Q&As · Created by Pranav Gawai (Part 1)

Q: What is the difference between a process and a thread, and how does the OS handle them differently?

"A process is a completely isolated program instance with its own private address space containing code, data, heap, and file descriptors. Threads, on the other hand, are lightweight execution units that exist inside a process and share the parent's memory heap, code, and OS descriptors, though they each maintain a private stack and program counter. The OS handles processes by giving them strong isolation boundaries, meaning if one process crashes, it won't affect others, but communicating between them requires expensive system calls like IPC or shared memory. Threads share memory natively, which makes communication extremely fast, but a crash in one thread can crash the entire process, and they require synchronization like mutexes to prevent race conditions."

KEYWORDS: ISOLATED ADDRESS SPACE SHARED HEAP PRIVATE STACK IPC OVERHEAD SYNCHRONIZATION

Follow-Up: What is a thread pool? How does copy-on-write optimize process fork?

Common Mistake: Stating that threads share stacks. Each thread has its own private execution stack.

Expected Depth: Detail how virtual page directory mappings differ between process and thread switches.

Q: Explain the difference between a Mutex and a Semaphore. When would you use one over the other?

"A Mutex is a locking mechanism designed strictly for mutual exclusion. It has a concept of ownership, meaning only the specific thread that acquired the lock can release it. A Semaphore, on the other hand, is a signaling mechanism that manages access to a pool of resources using an integer counter. It doesn't have ownership, so any thread can signal or wait on it. I would use a Mutex when I need to protect a shared variable or critical section of code from concurrent writes by multiple threads. I would use a counting semaphore when I have a finite pool of resources, like a database connection pool, where I want to allow up to 'N' threads to access the resources simultaneously, blocking the N+1th thread."

KEYWORDS: LOCK OWNERSHIP SIGNALING MECHANISM MUTUAL EXCLUSION RESOURCE POOL THREAD SYNCHRONIZATION

Follow-Up: What is a binary semaphore? What is priority inversion?

Common Mistake: Releasing a Mutex from a different thread than the one that locked it.

Expected Depth: Explain spinlocks vs blocking mutexes and their CPU consumption trade-offs.

Top Expected Interview Questions

Placement Q&As · Created by Pranav Gawai (Part 2)

Q: What is a Deadlock, and what are the four conditions required for a deadlock to occur?

"A deadlock is a state where a set of processes are permanently blocked because each process holds a resource and is waiting for another resource held by another process in the same set. For a deadlock to happen, all four Coffman conditions must hold simultaneously. First, Mutual Exclusion, meaning the resources cannot be shared. Second, Hold and Wait, where a process must be holding at least one resource while waiting to acquire another. Third, No Preemption, meaning resources cannot be forcibly taken from a process. And fourth, Circular Wait, where a closed loop of processes exists such that each process waits for a resource held by the next. If we can break even one of these four conditions—like imposing a global resource ordering to prevent circular wait—deadlocks can be completely prevented."

KEYWORDS: COFFMAN CONDITIONS MUTUAL EXCLUSION HOLD & WAIT CIRCULAR WAIT RESOURCE ORDERING

Follow-Up: Differentiate deadlock prevention vs avoidance. How does Banker's algorithm work?

Common Mistake: Confusing deadlock prevention with deadlock detection. Prevention prevents the state, detection resolves it after it occurs.

Expected Depth: Detail the Banker's algorithm safety check time complexity and its practical limitations in modern OS.

Q: Differentiate Paging from Segmentation. How do they handle memory fragmentation differently?

"Paging is a memory management scheme where virtual memory and physical memory are divided into fixed-size blocks called pages and frames. It completely eliminates external fragmentation because any free physical frame can be allocated to any process page, though it can cause minor internal fragmentation in the very last page of a process. Segmentation is a programmer-centric scheme where memory is divided into variable-sized logical segments, like code, stack, or heap. Because segment sizes vary, loading and unloading them leaves variable-sized holes in physical memory, which leads to external fragmentation over time. Modern systems often use a hybrid called paged segmentation to get the logical benefits of segments while using paging internally to prevent external fragmentation."

KEYWORDS: FIXED-SIZE PAGES VARIABLE-SIZE SEGMENTS INTERNAL FRAGMENTATION EXTERNAL FRAGMENTATION PAGED SEGMENTATION

Follow-Up: What is the purpose of TLB? What is Belady's Anomaly?

Common Mistake: Saying paging causes external fragmentation. Paging ONLY causes internal fragmentation.

Expected Depth: Explain multi-level page table lookups and how TLB reduces memory reference overhead.

OS Rapid Fire Questions

Fast-Recall Flashcards for Last-Minute Self-Testing

Q: What is a PCB?

A: Process Control Block; a kernel data structure storing process state.

Q: What is the PID of the init process?

A: PID 1, which adopts orphan processes.

Q: What is Belady's Anomaly?

A: FIFO page replacement causing more page faults as frame counts increase.

Q: Name a scheduling algorithm that prevents starvation.

A: Round Robin (time-sliced scheduling).

Q: What is a translation lookaside buffer?

A: TLB is a fast hardware cache for virtual-to-physical address translations.

Q: Difference between user mode and kernel mode?

A: User mode is restricted; kernel mode has full hardware access.

Q: What system call is used to create a process in Unix?

A: `fork()` (duplicates the parent process).

Q: What is a spinlock?

A: A lock where threads poll in a loop (busy-wait) until the lock is available.

Q: What is thrashing?

A: The system spends more time swapping pages in/out of disk than executing work.

Q: What is Peterson's solution?

A: A software-based algorithm that solves the critical section problem for two processes.

Q: What is the Banker's algorithm used for?

A: Deadlock avoidance by checking if resource allocation keeps the system in a safe state.

Q: What is an inode?

A: Index node; a filesystem structure storing file metadata, size, permissions, and block pointers.

OS Common Traps & Deflections

Tactical Responses to Deflect Tricky Placement Questions

Trap 1: The Multi-core Spinlock Illusion

Interviewer: "Why is a spinlock highly efficient on a single-core processor?"

Deflection: Correct the interviewer: Spinlocks are actually **terrible** on a single-core CPU. A thread waiting on a spinlock will spin in a loop, wasting the entire CPU time slice, and since there is only one core, the thread holding the lock cannot execute to release it. Spinlocks are only useful on multi-core systems where one core spins while another releases the lock.

Trap 2: The Fork Heap Sharing Myth

Interviewer: "Since `fork()` creates a duplicate process, does it immediately copy the entire parent RAM heap?"

Deflection: Clarify that `fork()` uses Copy-on-Write (COW). It does not copy the RAM immediately; instead, it duplicates the page table pointers, marking them read-only. Both parent and child share the same physical memory until one of them writes, at which point a page fault occurs and a duplicate frame is created.

Trap 3: The Starvation-Free CPU Scheduler

Interviewer: "Does Shortest Job First (SJF) guarantee that no process will starve?"

Deflection: State that SJF actually has a **high** risk of starvation for long processes. If a steady stream of short processes enters the queue, a long process will wait indefinitely. To prevent starvation, we must implement aging to increase process priority over time.

Trap 4: The TLB Miss Page Table Flush

Interviewer: "When a TLB miss occurs, does the OS kernel immediately flush the entire TLB?"

Deflection: Explain that a TLB miss simply means the address translation isn't in cache. The CPU performs a page table walk to find it, caches it in the TLB, and resumes. Flushes only happen during a context switch to a different process (unless ASIDs are used) or when page tables are modified.

Trap 5: The Thread Crash Isolation

Interviewer: "If a thread crashes due to a segmentation fault, does the rest of the process continue running?"

Deflection: No. Since threads share the same address space, a segmentation fault signals the OS that memory corruption has occurred. The OS sends a SIGSEGV signal to the entire process, terminating all of its sibling threads immediately. Threads lack fault isolation.

Personal Notes & Scribbles